

MS SQL- PART 2

Linked Servers on MS SQL Part 2

By Don Schlichting

Introduction

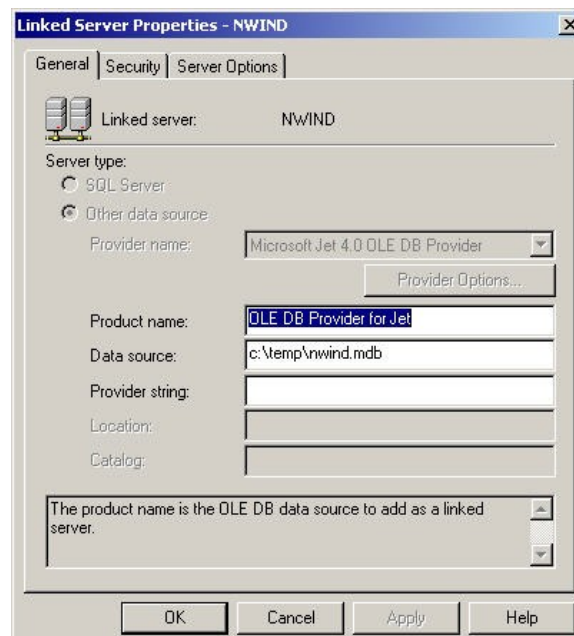
In Part 1 of this series, we reviewed the purpose and reasons for using linked servers, as well as when not to use them. The first linked server created was to an Excel sheet containing data from the Authors table of the SQL Pubs database. We will now move on to other data sources and security.

Access

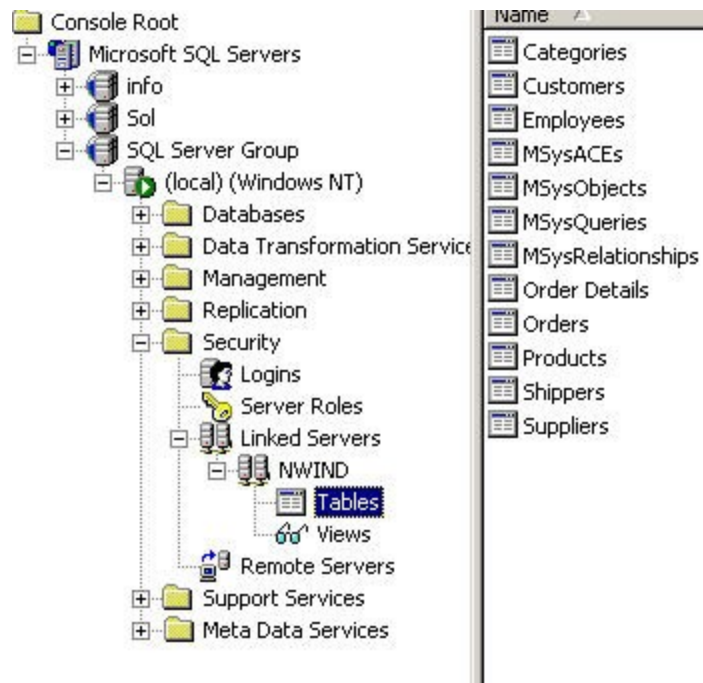
Northwind , the sample Access database we will be using, can be downloaded at <http://www.microsoft.com/downloads/details.aspx?FamilyID=C6661372-8DBE-422B-8676-C632D66C529C&displaylang=en> . Save this file into your c:\temp directory and expand it. Convert it to your correct version of Access. For these examples, Access 2000 SR 1 was used. If the file was successfully expanded, the Northwind Traders splash will appear when the database is opened.

To create the linked server, open Enterprise Manager and navigate to Security, right click on Linked Servers, and select New Linked Server.

In the Linked Server Properties box, select Microsoft Jet 4.0 OLE DB Provider as the provider name. The product name will be "OLE DB Provider for Jet" and the data source will be "c:\temp\nwind.mdb." Click OK to finish.



A linked server called NWIND should now appear under the Linked Servers heading. Clicking the "Tables" icon will list the table names of the Access database.



To create the link with TSQL, open Query Analyzer, enter and run:

```
exec sp_addlinkedserver
    @server = 'Nwind2',
    @provider = 'Microsoft.Jet.OLEDB.4.0',
    @srvproduct = 'OLE DB Provider for Jet',
    @datasrc = 'c:\temp\nwind.mdb'
```

This should return (1 row(s) affected) lines.

To verify that the linked server was successfully created, enter the select:

```
SELECT *
FROM NWind2...Orders
```

The select should return all the rows from the Orders table:

<pre>SELECT * FROM NWind2...Orders</pre>			
	CustomerID	EmployeeID	Freight
1	VINET	5	32.380
2	TOMSP	6	11.610
3	HANAR	4	65.830
4	VICTE	3	41.340

To get a quick list of your servers while in the master database, execute:

```
SELECT *
FROM sys.servers
```

<pre>SELECT * FROM sys.servers</pre>					
	srvid	srvstatus	srvname	srvproduct	providername
1	0	1089	DONS13	SQL Server	SQLOLEDB
2	1	1184	NWIND	OLE DB Provi...	Microsoft.Jet
3	2	1184	NWind2	OLE DB Provi...	Microsoft.Jet

Our two linked servers will appear with a Jet provider name.

MS SQL

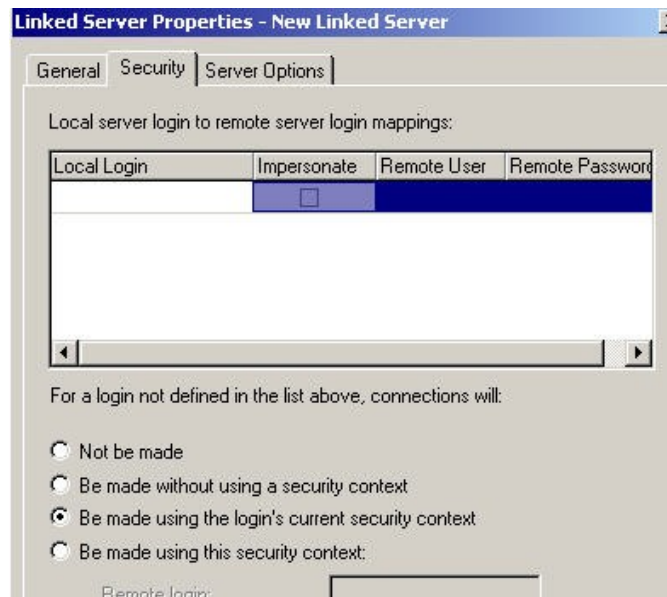
Linked Servers on MS SQL Part 2 - Page 2

By Don Schlichting

SQL

In this first SQL linked server example, two servers and a windows administrator account are required. From inside Enterprise Manager, Security, Linked Servers, we will again right click to create a new Linked Server. For the linked server name, enter the SQL name of your second server, and then click the server type "SQL Server" radio button. This will gray out the other data source information boxes we used previously.

Now click the Security tab and select the "Be made using the login's current security context."



Click OK to save the new linked server. This security context will pass your current login information to the remote server. The new server should now appear in the Linked Server list. Clicking the Tables icon will display the remote servers Master table list.

Direct Reference

To use the new connection, open Query Analyzer using Windows Authentication. Your windows account must have rights to the remote servers SQL database. From inside Query Analyzer, issue a select to verify connectivity.

```
SELECT *  
FROM infonet.northwind.dbo.orders
```

The four-part name includes the Link Name, Database Name, Owner and Object. (For more on four part naming, see Part 1)

The Orders table will be returned:

<pre>SELECT * FROM infonet.northwind.dbo.orders</pre>				
	OrderID	CustomerID	EmployeeID	OrderDate
1	10248	VINET	5	1996-07-04
2	10249	TOMSP	6	1996-07-05
3	10250	HANAR	4	1996-07-06
4	10251	VICTE	3	1996-07-07
5	10252	SUPRD	4	1996-07-08
6	10253	HANAR	3	1996-07-09

We can tie our remote server to our local server by using standard joins. This example will join the local Northwind Orders Detail table to the remote Products table. (Change Query Analyzer to be in the Northwinds database.)

```
SELECT loc.OrderID, loc.ProductID, rmt.ProductName
FROM [Order Details] loc INNER JOIN Infonet.Northwind.dbo.Products rmt
ON loc.ProductID = rmt.ProductID
```

<pre>SELECT loc.OrderID, loc.ProductID, rmt.ProductName FROM [Order Details] loc INNER JOIN Infonet.Northwind.dbo.Products rmt ON loc.ProductID = rmt.ProductID</pre>			
OrderID	ProductID	ProductName	
1	10285	Chai	
2	10294	Chai	
3	10317	Chai	
4	10348	Chai	
5	10354	Chai	
6	10370	Chai	

We will call the query just issued a "Direct Reference." Four-part naming is used to call the remote reference directly. Four-part naming creates very clean code, easy to follow and create. Unfortunately, this type of query will execute on our local server. Meaning all of the rows will be required to travel across the wire from the remote server to the local server for processing. Not a problem for small retrievals, but may have a negative performance impact with large remote objects.

Open Query

For large remote objects, the solution to performance problems is the OPENQUERY function. With OpenQuery, our first select example would read:

```
SELECT *
FROM OPENQUERY(InfoNet, 'SELECT * FROM Northwind.dbo.Orders')
```

InfoNet is the name of the remote server, followed by the command. The same results are returned as our four-part naming example, but now the processing is being done on the remote server.

In the Join example, the four-part naming changes from:

```
SELECT loc.OrderID, loc.ProductID, rmt.ProductName
FROM [Order Details] loc INNER JOIN Infonet.Northwind.dbo.Products rmt
    ON loc.ProductID = rmt.ProductID
```

To:

```
SELECT loc.OrderID, loc.ProductID, rmt.ProductName
FROM [Order Details] loc INNER JOIN
OPENQUERY(InfoNet, 'SELECT * FROM Northwind.dbo.Products') rmt
    ON loc.ProductID = rmt.ProductID
```

Drop Server

To remove our linked server execute:

```
sp_dropserver 'INFONET', 'droplogins'
```

The 'droplogins' option will remove any and all logins associated with the linked server.

To drop from Enterprise Manager, right click the server name and select Delete. A dialog box will prompt you to remove any logins.

sp_addlinkedserver

To recreate our link from inside Query Analyzer, enter and run:

```
EXEC sp_addlinkedserver
    @server = 'TEST',
    @srvproduct = 'SQLServer OLEDB Provider',
    @provider = 'SQLOLEDB',
    @datasrc = 'InfoNet'
```

@server will be the name of the new link. This does not have to be the same as your actual data source. Although Query Analyzer does not force it, UPPER CASE should always be used for the server name. Enterprise Manager does this automatically.

An additional parameter that can be passed is @catalog. The catalog will be the default database for the Linked Server. If we change our example above to the following:

```
EXEC sp_addlinkedserver
    @server = 'TEST',
    @srvproduct = 'SQLServer OLEDB Provider',
    @provider = 'SQLOLEDB',
    @datasrc = 'InfoNet',
```

@catalog = 'Northwind'

Our table view in Enterprise Manager now shows the Northwind tables rather than Master:



The security context of "Be made using the login's current security context" will be automatically assigned.

OpenRowset

For one time and ad hoc uses, the OPENROWSET function can be used rather than creating a linked server. The function receives all the parameters necessary to create the connection on the fly.

```
SELECT *  
FROM OPENROWSET('SQLOLEDB', 'INFONET' ; 'sa' ; 'password',  
  'SELECT * FROM northwind.dbo.orders' )
```

Its end result can be used just like a linked server.

Conclusion

Linked servers can provide effective and powerful connection options. Functions and tools provide many features and choices. In Part 3 we'll explore additional security options and connectivity to Oracle.